

Fly GACA — لوحة المدرّب — مواصفة التصميم

القسم operations-it-06 النوع spec الحالة active المالك Founder آخر تحديث 21-06-2026

الحالة: تصميم، غير مبني بعد · يرتبط بـ: خطة العمل 5.2§، `roadmap.md` المرحلة الخامسة أُعِدَّ في: 25-05-2026

هذه مواصفة بناء لوحة المدرّب/المدرسة بين الشركات (B2B). وهي عمداً قائمة على المواصفة أولاً: فالجزء الصعب ليس واجهة المستخدم، بل نموذج بيانات متعدّد المستأجرين وقواعد الأمان التي تتيح لمستخدم أن يرى بيانات آخر دون إضعاف العزل الصارم لكل مستخدم القائم على المنصة اليوم. وهذا يجب أن يُصمّم لا أن يُستعجل.

يُسوَّق `schools.html` هذا أولاً — "لوحة مشرف المدرسة ... جاهزية الفوج للاختبار وتقدّم كل طالب على حدة ... مُتضمّنة مع كل عقد" وتبني هذه المواصفة ذلك الوجود.

1. القيد الجوهري

النموذج اليوم عزل صارم لكل مستخدم. تقول `firestore.rules` ذلك صراحةً: "لا يوجد مسار يستطيع به مستخدم أن يقرأ ملف مستخدم آخر أو سجلّه" وكل قاعدة هي `isOwner(vid)`.

تتطلّب لوحة المدرّب قراءات عابرة بين المستخدمين مُتحكّماً بها — مدرّب يرى تقدّم طالب. ويقوم التصميم الكامل أدناه على فعل ذلك بأمان. فإن أخطأت في القواعد تسرّبت سجلّات طالب؛ وذلك أكبر خطر منفرد وهو سبب كون هذه المواصفة قائمة على المواصفة أولاً.

قرار التصميم المحوري الذي يحلّه: لا يقرأ المدرّبون أبداً داخل `users/{cadetUId}`. بل يكتب عميل كل طالب ملخص تقدّم محصور النطاق في الشجرة الفرعية الخاصة بالمدرسة (`schools/{id}/roster/{cadetUId}`). ويقرأ المدرّبون مستندات المدرسة، لا مستندات الطلاب أبداً. ويبقى عزل `users/{uid}` سليماً 100%.

2. الأدوار

الدور	من	القدرة
الطالب / الطيار	موجود اليوم — <code>users/{uid}</code>	يملك بياناته؛ ويختار الانضمام إلى فوج
المدرّب	جديد	يقرأ تقدّم فوجه؛ ويعيّن مسارات قراءة
مشرف المدرسة	جديد	يدير المقاعد والمدرّبين والقائمة؛ ويرى ملخص الفوج

الطالب على خطة مدرسة موسوم أصلاً: `users/{uid}.entitlement.schoolId` موجود في النموذج الحالي (`store.js`، عند `entitlement.source === 'school'`). وهذا الحقل هو المركز لكل ما هنا.

3. الموافقة والخصوصية — من الدرجة الأولى

سجل الطالب وبيانات دراسته بيانات شخصية. وتقييم أثر حماية البيانات (DPIA) بموجب PDPL بوابة إطلاق أصلًا (`roadmap.md`)؛ ويجب أن تكون الرؤية العابرة بين المستخدمين ضمن نطاق ذلك التقييم.

قاعدتان تلتزم بهما هذه المواصفة:

1. النطاق. يرى المدرّسون بيانات الدراسة وجاهزية الاختبار فقط — تقدّم المدرسة الأرضية، ودرجات الاختبارات التجريبية، وأعلام الحداثة. ولا يرون السجل الشخصي. فالسجل سجل الطيار الخاص؛ أبقيه خاصًا. (قرار مفتوح — 10§ — لكن هذا هو الافتراضي الموصى به.)

2. موافقة صريحة قابلة للإلغاء. عند تسجيل طالب في فوج يرى شاشة موافقة تسمّي بدقّة ما سترام المدرسة، ويجب أن يقبل. ويمكنه إلغاؤها من الإعدادات، فيقلب ذلك مدخل قائمته إلى `consent: false` ويفرغ الملخص المشترك.

4. نموذج البيانات (Firestore)

شجرة `schools` جديدة من المستوى الأعلى. لا يتغيّر شكل أي شيء تحت `users/{uid}`؛ وتُضاف مجموعة فرعية جديدة واحدة هناك (ملخص التقدّم، وهو مفيد بذاته أيضًا).

```
schools/{schoolId}
  name          string
  adminUids      string[]      // مشرفو المدرسة
  instructorUids string[]      // المدرّسون
  seatLimit      number        // المقاعد المشتراة
  seatsUsed      number        // تتعهّده دالة سحابية
  createdAt/updatedAt timestamps

schools/{schoolId}/roster/{cadetUid}
  // حقول التسجيل - تُكتب فقط بواسطة دوال السحابة:
  cadetName      string        // غير معيارية لقائمة الفوج
  cadetEmail      string
  status          'invited' | 'active' | 'removed'
  consent         boolean      // قَبِل الطالب موافقة مشاركة البيانات
  enrolledAt      timestamp
  // ملخص التقدّم - يكتبه عميل الطالب نفسه (انظر 6§):
  readiness       number        // نسبة جاهزية الاختبار %, 100-0
  lastMockScore   number
  lessonsDone     number
  currencyFlags   map           // { day:'ok', night:'due', ifr:'lapsed' }
  progressUpdatedAt timestamp
  // المدرّب ← الطالب، يكتبها المدرّسون:
  assignedPaths   string[]      // معرّفات مسارات القراءة التي يقترحها المدرّب

users/{uid}/progress/summary      // مستند التقدّم من جانب الخادم
  readiness, lastMockScore, lessonsDone, currencyFlags, updatedAt
```

المتطلب المسبق — المرحلة A. تقدّم الدراسة حاليًا ليس في Firestore — يحفظ `store.js` الملف الشخصي والسجل فقط؛ وتعيش درجات الاختبارات التجريبية وتقدّم الدروس في `localStorage` (`study-progress.js`). ولكي يرى المدرب أي شيء، يجب أن يكون تقدّم الطالب من جانب الخادم. لذا تضيف المرحلة A `users/{uid}/progress/summary` ، يكتبه عميل الطالب. ثم يعكس عميل الطالب أيضًا ذلك الملخص في مداخله `schools/{id}/roster/{cadetUid}` . تتحقّق من الحفظ الحالي قبل البناء.

5. قواعد الأمان — مسوِّدة أولية

هذه مسوّدة أولية ويجب اختبارها بالوحدة في محاكي Firebase قبل أي نشر. وهي توسّع `firestore.rules` اليوم (تبقى كتلة `users/{uid}` دون تغيير).

```
match /schools/{schoolId} {  
  function school() {  
    return get(/databases/${database}/documents/schools/${schoolId}).data;  
  }  
  function isAdmin() {  
    return signedIn() && request.auth.uid in school().adminUids;  
  }  
  function isStaff() {  
    return signedIn() && (request.auth.uid in school().adminUids  
      || request.auth.uid in school().instructorUids);  
  }  
  
  // مستند المدرسة: يقرأه الموظفون؛ ويحرّر المشرفون الحقول غير البنيوية.  
  // الإنشاء وعدد المقاعد لدوال السحابة فقط .  
  allow read: if isStaff();  
  allow update: if isAdmin() && onlyAllowedSchoolFields();  
  allow create, delete: if false;  
  
  match /roster/{cadetUid} {  
    function roster() { return resource.data; }  
    // يرى الموظفون كامل الفوج؛ ويرى الطالب مدخله وحده .  
    allow read: if isStaff() || isOwner(cadetUid);  
    // قد يحدث الطالب حقول ملخص تقدّمه وحدها ، وفقط .  
    // ولا يمكنه أبدًا لمس حقول التسجيل. true ما دامت موافقته .  
    allow update: if isOwner(cadetUid)  
      && roster().consent = true  
      && onlyProgressFields();  
    // وحدها assignedPaths قد يحدث المدرب .  
    allow update: if isStaff() && onlyAssignedPaths();  
    // التسجيل / الإزالة لدوال السحابة فقط (يحرّك مقعدًا أيضًا )  
    // users/{uid}.entitlement.schoolId). وبمضبط .  
    allow create, delete: if false;  
  }  
}
```

فحوص فرق — يجب أن تكون `onlyAllowedSchoolFields() / onlyAssignedPaths() / onlyProgressFields()` مجموعة جزئية من المجموعة المسموح بها. اكتبها صراحةً؛ فمنا يختص التسريب.

لماذا هذا آمن: لا يقرأ المدرّبون إطلاقاً سوى `schools/{id}/...`. ولا تزال لا توجد قاعدة في أي مكان تتيح لمستخدم أن يقرأ شجرة `users/{uid}` لمستخدم آخر. والسطح العابر بين المستأجرين مجموعة واحدة، بمجموعة قواعد محكمة قابلة للاختبار.

6. تدقيق التسجيل والموافقة

1. إنشاء المدرسة — بواسطة Fly GACA عند توقيع العقد (دالة سحابية / أداة مشرف)، لا خدمة ذاتية. تضبط

`name` `adminUids` `g` `seatLimit`.

2. يضيف المشرف المدرّبين — بالبريد الإلكتروني؛ تحلّ دالة سحابية البريد إلى `uid` وتلحقه بـ `instructorUids`.

3. تسجيل الطلاب — يُدخل المشرف بريد الطلاب الإلكتروني (موصى به) أو يشارك رمز انضمام للمدرسة. وفي كلتا الحالتين تنشئ دالة سحابية مدخل `roster/{cadetUids}` بـ `status: 'invited'`، `consent: false`.

4. يوافق الطالب — عند تسجيل الدخول التالي يرى الطالب شاشة موافقة: "ستطلع مدرسة الطيران [name] على تقدّمك في المدرسة الأرضية ودرجات اختباراتك التجريبية. ولن تطلع على سجلّك. ويمكنك إلغاء ذلك في أي وقت من الإعدادات" وعند القبول ← تضبط دالة سحابية `status: 'active'`، `consent: true`، وتزيد `entitlement.schoolId`، `seatsUsed`.

5. الإلغاء — الإعدادات ← "مغادرة فوج المدرسة" تضبط `consent: false` ويفرغ العميل حقول الملخص المشترك.

تمرّ كل كتابات المقاعد/الاستحقاق/بنية القائمة عبر دوال السحابة (Admin SDK) — تمامًا كما يُعالج `entitlement` اليوم.

7. خطة واجهة المستخدم

صفحة جديدة `instructor.html` (مبوبة لـ `isStaff`)، إضافة إلى سطح موافقة الطالب.

- عرض الفوج — جدول/بطاقات: اسم كل طالب، ونسبة جاهزية الاختبار %، وآخر درجة تجريبية، والدروس المكتملة، وأعلام الحداثة، وآخر نشاط. الفرز + التصفية (مثل "أرني من تأخّر"). وهذا هو "اطلع بلمحة على من هو جاهز ومن يتخلّف" الموعود في `schools.html`.
 - التعمّق لكل طالب — اتجاه جاهزية الطالب، وتاريخ الاختبارات التجريبية، وخريطة دروس المدرسة الأرضية، والحداثة. إضافة إلى عنصر تحكّم تعيين مسارات قراءة ← يكتب `assignedPaths`؛ ويراه الطالب كمسارات مقترحة.
 - عرض المشرف — المقاعد المستخدمة / الإجمالي، إضافة/إزالة الطلاب والمدرّبين، وملخص جاهزية الفوج (المتوسط، التوزيع، عدد من لم يبدأ بعد).
 - شاشة موافقة الطالب — تُعرض مرّة واحدة عند التسجيل؛ وتُلخّص مجددًا في الإعدادات.
- أعد استخدام `curr-card` و `stat-card`. الموجودة وعناصر إطار صفحة الحساب و `gate.js`.

8. مراحل البناء

المرحلة	النطاق	الجهد
A	ملخص تقدم من جانب الخادم (<code>users/{uid}/progress/summary</code>) يكتبه عميل الطالب — مفيد بذاته	S-M
B	نموذج بيانات <code>schools</code> + <code>roster</code> + قواعد الأمان + اختبارات المحاكى	M
C	دوال السحابة: إنشاء مدرسة، إضافة مدرّج، تسجيل طالب، موافقة، إلغاء، محاسبة المقاعد	M
D	<code>instructor.html</code> — عرض الفوج + التحقق لكل طالب	M
E	إدارة مقاعد المشرف + تعيين مسارات القراءة	M

الإجمالي $L \approx$ (بناء مركز يمتدّ عدّة أسابيع، منفرد) — يطابق تقدير خطة العمل. المرحلتان A و B هما الأساس ويجب إنزالهما أولاً وبشكل صحيح؛ والمرحلة D هي الجزء الوحيد الذي يراه المستخدم.

9. القرارات المفتوحة (أنت)

1. رؤية السجل — يرى المدرّجون بيانات الدراسة فقط (موصى به)، أم السجل أيضًا؟ الخصوصية تقول الدراسة فقط.
2. التسجيل — إدخال المشرف للبريد الإلكتروني (تحكم أكبر، موصى به) مقابل رمز انضمام (احتكاك أقل، تحكم أضعف)؟ يمكن دعم كليهما.
3. إنشاء المدرسة — Fly GACA عند العقد (موصى به لمنتج B2B مدفوع) مقابل تسجيل بخدمة ذاتية.
4. DPIA — يجب إضافة مشاركة البيانات العابرة بين المستخدمين إلى نطاق تقييم أثر حماية البيانات بموجب PDPL. وهذه بوابة إطلاق؛ ولا يمكن إطلاق لوحة المدرّج حتى تُغطى.

10. المخاطر

- قواعد الأمان. خطأ في قاعدة يسرّب بيانات طالب. التخفيفات: تصميم عدم القراءة داخل `users/{uid}` أعلمه؛ وفحوص فرق `affectedKeys()` الصريحة؛ واختبارات قواعد محاكي Firebase الإلزامية؛ والطرح المتدرّج بمدرسة تجريبية واحدة.
- PDPL. الرؤية العابرة بين المستخدمين هي بالضبط ما يدقّقه تقييم أثر حماية البيانات (DPIA) — لا تنشر قبل أن تُغطى.
- الملّخصات المتقدمة. يكتب عميل الطالب ملّخص التقدّم؛ فإن تقادم رأى المدرّج بيانات قديمة. التخفيف: اكتب الملّخص عند نهاية كل جلسة دراسة، وأظهر `progressUpdatedAt` في واجهة المستخدم بحيث يكون التقادم مرئيًا.
- تضخم النطاق. قاوم المراسلة والتقييم والحضور والفوترة في الإصدار الأول. الإصدار الأول = رؤية جاهزية الفوج. وذلك وحده يبيع المقاعد.